Workshop Parts Desk - Final Team Guide

HackMatrix 1.0 (KSIT, Agentic AI) - team F60 - final model v3.3 + gateway v3. Status 5 Nov 2026, 18:50 IST.

No API keys or tokens are inside this PDF. It only says where they are kept. Never paste keys or the gateway token into chats, slides, screenshots or GitHub.

1. The 30-second summary

- We built an **AI agent** (one Python file, `solution.py`) that runs a workshop parts counter by itself.
- For each batch of 6 requests and 12 power-supply parts it either **dispatches** a matching part or **escalates** to a human, using a small tool budget.
- The AI "brain" is reached through **our own gateway** on **Cloudflare Workers** (free), which calls **Groq** and **Gemini** (free).
- **Final setup:** controller **v3.3** + gateway **v3** (portal gateway revision 3) - **nominated**.
- **Results:** pre-twist official 100/100, post-twist official **100 / 100** (routine and demanding), **0 unsafe dispatches ever**. Leaderboard: 20 + 30 = 50 of 50 points so far.

Item	Status
Controller	v3.3 - post-twist official 100/100, deterministic, Python standard library only
Gateway in use	parts-gateway-v3 on Cloudflare = portal revision 3, connection verified
Old gateway	parts-gateway = revision 1 (used by early uploads) - keep it running
Nominated	v3.3 post-twist official (fingerprint starts 0f0ebb2f) - used for the final private test (50%)
Final test	Run automatically by organisers on private batches after cutoff

2. The problem in simple words

A workshop gets parts requests written in everyday language, e.g. *"I need 12 V through USB-C, between 10 and 20 W"*. There are 12 power supplies, each described in text. For every request the agent must **dispatch** a matching part or **escalate** to a human. One **wrong** part makes the whole batch score zero.

Tools (14 units shared by all 6 requests; spend 13 or fewer)

Tool	Cost	What it gives
inspect	1	Part spec card + stock records (after the twist: exact stock if the desk has the latest warehouse record).
verify	2	Part spec card + the TRUE current warehouse stock.
clarify	2	Asks the customer. After the twist it returns the full approved request including the watt range.
dispatch / escalate	0	Final decision for one request. Irreversible.

Scoring per batch (out of 100)

Part	Max	How to earn it
Outcome	80	Right part sent, or escalated when no part could serve it. Escalating a solvable request gives only 15%.
Integrity	15	Every request gets a decision.
Efficiency	5	Spend 13 units or fewer (all-or-nothing). Spending all 14 loses it.

Overall marks: 20% pre-twist validation + 30% post-twist validation + 50% final private test.

3. What the twist changed (and how we handled it)

Twist rule	What it means	What v3.3 does
Watt RANGE	Part watts must be inside min..max. A bigger part is no longer always OK.	Dispatches only inside the range. No max stated = exact match only (safe).
"Contact the customer for limits"	The range is unknown until clarify.	Never dispatches before clarify; reads the full reply.
Stock documents + manifest	inspect gives stock documents; exact only if revision = warehouse_revision. Old "available" field is misleading.	Reads the exact quantity, ignores the legacy field.
Desk behind warehouse	desk_receipt_revision < warehouse_revision: inspect cannot be exact.	Skips inspect and goes straight to verify.
Shared stock	A part can hold 2+ units for several requests.	Lets one part serve several requests (this fix took "changed" from 87.8 to 100).
AI byte budget	Max 48,000 bytes of AI messages per batch.	Uses at most ~4,800 bytes.

4. How our agent works (5 steps)

Step	What happens
1. Read the right version	Latest approved revision with issued_at <= as_of. Drafts and late "retry" copies are ignored.
2. Two independent readers	A rule-based parser and the AI both extract voltage, connector and watt range. Trusted only if they agree.
3. Second opinion	If they disagree, the AI is asked again. Accepted only if it agrees with the parser, or with its first read and every number appears in the text.
4. Planner	For each possible inspect / verify / clarify it computes how much the expected score rises per unit, picks the best, repeats. Hard stop at 13 units.
5. Safety gate	Best assignment of parts to requests (respecting stock counts). Before every dispatch, code checks voltage, connector and range.

The flow in one line

```
documents -> latest approved text -> parser + AI agree? -> planner buys useful checks -> safety gate -> dispatch or escalate
```

Why these choices

- **Facts separate from decisions:** the AI reads text; plain code decides. An AI guess can never cause a wrong dispatch on its own.
- **Tool card is final for parts:** inspect/verify return the official spec, so unclear part descriptions are confirmed before use.
- **Expected-value planning:** it does not check everything (too expensive) or nothing (risky) - only what changes a decision.
- **Deterministic:** same input gives the same decisions every time, so results are reproducible and explainable.
- **Scalable:** exact search for small batches, fast greedy method for large ones (40 requests x 80 parts planned instantly).

5. Cloudflare and our gateway

What is Cloudflare?

Cloudflare is a large internet company that runs servers in hundreds of cities. It is best known for making websites fast and protecting them from attacks. **Cloudflare Workers** is its service for running small pieces of code on those servers, without managing any machine.

- **Always on:** no sleeping and no slow start-up, unlike many free hosts.
- **HTTPS for free:** every Worker gets an address like `https://name.account.workers.dev` on port 443.
- **Free plan:** about 100,000 requests per day, more than enough for us.
- **Secrets:** API keys are stored encrypted ("Secret" type) and never shown again, not even to us.
- **Browser only:** we created, edited and deployed everything from the dashboard - nothing installed.

Why we needed a gateway

The organisers run our code in a sandbox with **no internet**. When the agent needs the AI, their relay calls **our** HTTPS link. The gateway checks our bearer token, forwards the question to Groq/Gemini, and returns only the text. Keys never enter the ZIP.

	Old gateway (revision 1)	Gateway v3 (revision 3) - in use
Worker name	parts-gateway	parts-gateway-v3
URL	https://parts-gateway.yadhavan-periyannan.workers.dev/gateway	https://parts-gateway-v3.yadhavan-periyannan.workers.dev/gateway
Models	4, Gemini Flash first (only 20 calls/day free)	9, ordered by free daily quota
Extras	-	Race of 2 models, short timeouts, /health check
Rule	Keep running, do not edit	Keep running, do not edit (nominated)

v3 model order

Order	Model	Free quota (approx.)
Race (parallel)	Groq openai/gpt-oss-120b	~1000 / day
Race (parallel)	Gemini 3.5 Flash Lite	500 / day
Backups	Gemini 3.1 Flash Lite, Groq gpt-oss-20b, Groq qwen3.8-27b, Gemini Flash-Lite-latest	separate quotas
Last resort	Gemini 3.5 Flash, Gemini 3.8 Flash	20 / day each

Answers come back in 2-5 s (limit 45 s). Why v3: Gemini Flash hit its 20/day limit and caused a 503 error on the old gateway.

Where the secrets live (values NOT here)

Secret	Stored in	See it (driver Mac only)
GEMINI_KEY	Cloudflare secret (both Workers) + ~/hack/.env (as GEMINI_API_KEY)	cat ~/hack/.env
GROQ_KEY	Cloudflare secret (both Workers) + ~/hack/.env (as GROQ_API_KEY)	cat ~/hack/.env
GATEWAY_TOKEN	Cloudflare secret + portal (hidden) + ~/hack/.gateway_token	cat ~/hack/.gateway_token

6. Files

File	Details
submission_v33.zip (FINAL)	Contains ONLY solution.py at the ZIP root. ~32 KB. Portal fingerprint starts 0f0ebb2f.
solution.py (v3.3)	decide(observation, state) returns exactly {"action", "state"}. Imports only json, re, itertools. Python 3.12. No keys, no network code.
~/hack/kit/	Organisers' initial packet (rules, simulator, local_runner.py, data) used for local testing
~/hack/worker.js, worker_v3.js	Gateway code for the old and v3 Workers
~/hack/kit/gwtest.py, gwtest3.py, gwdebug.py	Gateway speed / stress / debug tests
~/hack/.env, ~/hack/.gateway_token	Secrets - never share, never upload
Older ZIPs (submission.zip, _v2, _v3_twist, _v31, _v32)	Earlier versions, kept for reference

7. Submission history

Time	Stage / type	Version	Gateway	Result
12:59	Pre-twist practice	v1	rev 1	100
13:04	Pre-twist official	v1	rev 1	100
~15:00	Pre-twist practice	v2	rev 1	100 (after infrastructure retry)
~15:35	Pre-twist official	v2	rev 1	100
~16:15	Pre-twist official	v2	rev 3	100 (nominated before twist)
16:45	Post-twist auto re-run	v2	rev 3	51.3 / 39.2 (old rules unsafe now)
~17:30	Post-twist practice	v3	rev 3	94.8 / 100
~17:40	Post-twist official	v3	rev 3	100 / 87.8
~18:15	Post-twist practice	v3.3	rev 3	94.8 / 100
~18:35	Post-twist official	v3.3	rev 3	**100 / 100 - NOMINATED**

Scores after the twist are shown as routine / demanding ("ordinary / changed").

8. How we tested

Test	Result
52 public pre-twist batches (perfect-reader model)	v2: 99.3, 0 unsafe
New wording ("Type C port", "DC jack", "VDC")	v2: 98.1 (v1 dropped to 51.8)
Deliberately wrong AI on 25% of requests	0 unsafe - second opinion + "number must appear in text" check
Post-twist simulator we built (ranges, stock docs, desk lag, shared stock)	v3.3: 97.6, v3: 94.0, old v2: 37 with 44 unsafe
All 8 released twist records	Every request, part, stock document and clarify reply read correctly
Real gateway, 12 practice batches	100, 0 failures
Portal official post-twist	100 / 100

Ideas we tested and rejected (no gain): two-step planning, 36 setting combinations, cheap aged peeks, a more cautious stock prior, inspect-before-clarify.

9. Rules checklist (all passed)

Rule	Ours
Python 3.12, standard library + listed packages only	json, re, itertools
No HTTP/LLM SDKs in solution.py; AI only via op:llm	Yes
decide returns exactly action + state	Yes - organisers' local_check passed
ZIP: files at root, allowed types, no keys	One file: solution.py
1-8 model calls per batch; max 48,000 bytes total	1-2 calls, ~4,800 bytes max
Exact action keys (op + documented arguments only)	0 wrong-key actions in all tests
Gateway: HTTPS 443, bearer token, keys on our server	Cloudflare Workers + encrypted secrets
Gateway unchanged and online until final testing ends	Both Workers live and untouched

10. Presentation: slides and what to say

The separate file "HackMatrix_Presentation.pdf" has these 10 slides. Below is a simple script for each (about 20-30 seconds per slide, 4-5 minutes total).

Slide 1. Title

Say: We are team F60. We built an AI agent that runs a workshop parts counter safely, on a tight budget.

Slide 2. The problem

Say: Customers write requests in plain language. We have 12 parts and only 14 tool units for 6 requests. One wrong part makes the whole batch score zero, so safety comes first.

Slide 3. Why it is agentic

Say: The agent decides on its own which checks to run, reads the results, updates what it believes, and makes final decisions. It is not a chatbot; it acts.

Slide 4. How it works

Say: Five steps: read the right version, two readers must agree, second opinion when they disagree, a planner that buys only useful checks, and a safety gate before every dispatch.

Slide 5. Safety first

Say: The AI never decides alone. The AI and a rule-based parser must agree, part specs are confirmed by the official tool card, and code checks every dispatch. Result: zero unsafe dispatches in every test and every official run.

Slide 6. Smart budget

Say: For every possible check, the agent calculates how much it would raise the expected score per unit, and picks the best. It stops at 13 units to keep the efficiency bonus.

Slide 7. Gateway on Cloudflare

Say: The organisers' sandbox has no internet, so the agent calls our own gateway on Cloudflare Workers. It holds the keys safely and races free Groq and Gemini models, falling back across 9 models if one is busy.

Slide 8. Adapting to the twist

Say: After the twist, watts became a range, stock came as documents, and parts could be shared. Our old version dropped to about 51. We rebuilt the logic, tested it on all 8 released records and a simulator we wrote, and reached 100 on both groups.

Slide 9. Results

Say: Pre-twist official: 100. Post-twist official: 100 and 100. Zero unsafe dispatches. 50 out of 50 points so far.

Slide 10. What we would do next

Say: Learn stock reliability during a batch, smarter multi-step planning, and a paid model tier for higher limits. Thank you - happy to take questions.

11. Words you should know

Term	Simple meaning
Agent / agentic AI	A program that decides and acts by itself over several steps.
LLM	Large Language Model - the AI that reads text (Groq gpt-oss, Gemini).
API / API key	How programs talk to a service; the key is our account password for it.
Gateway	Our small web service that forwards the agent's questions to the AI.
Bearer token	A shared password in the request header so only the organisers can use our gateway.
HTTPS / port 443	Encrypted web connection on the standard secure port.
Cloudflare Worker	Free code running on Cloudflare's servers with an https://...workers.dev address.
Secret (Cloudflare)	An encrypted setting; the value can never be viewed again.
Rate limit (RPM / RPD)	Free plans allow only so many requests per minute / day.
Revision	A version of a document. Only the latest approved one counts.
Gateway revision	A saved gateway URL + token in the portal; each upload is locked to one.
Manifest	List of the latest warehouse and desk stock revisions per part.
Expected value	The average score you expect, weighting outcomes by probability.
Deterministic	Same input, same output, every time.
Nominate	Choosing which upload is used for the final private test.

12. Questions judges might ask

Q: What does your agent do?

A: Reads messy parts requests, spends a limited budget on the most useful checks, and sends a compatible in-stock part or hands the request to a human - never a wrong part.

Q: Why is it agentic?

A: It plans, uses tools, reads results, updates beliefs and makes irreversible decisions by itself under a budget.

Q: Where is the LLM used?

A: To extract voltage, connector and watt range from all requests and parts (one call per batch), plus a second-opinion call when it disagrees with the parser.

Q: Why not let the LLM decide?

A: One wrong guess zeroes a batch. The starter that did this scored about 34. We keep facts (AI + parser) separate from decisions (code).

Q: How do you guarantee safety?

A: Two readers must agree, tool cards confirm part specs, ranges are checked in code before every dispatch, and unknown ranges are clarified first.

Q: How did you handle the twist?

A: Read it from the new rules and 8 sample records, rewrote range matching, stock documents, desk lag and shared stock, tested on a simulator, and reached 100/100.

Q: Why did the first post-twist run score 51?

A: That was our pre-twist code re-run automatically under new rules. Its "bigger part is fine" logic was no longer valid.

Q: Why Cloudflare?

A: Free, always on, HTTPS on port 443 out of the box, encrypted secrets, and set up entirely in the browser.

Q: Why free models?

A: Extraction is a simple task and is double-checked by the parser, so free models are accurate enough. Nine models spread the rate limits.

Q: Is it scalable?

A: Exact search for small batches, fast greedy method for large ones; the gateway spreads load across many model quotas.

Q: Did you use AI tools to build it?

A: Yes, as allowed by rule 7. We tested every part and can explain each step.

Q: What would you improve?

A: Learn stock reliability within a batch, smarter multi-step planning, and paid model tiers for higher limits.

13. Until results are announced

Do	Do not
Keep both Cloudflare Workers running	Edit, delete or change the token of either Worker
Keep v3.3 nominated	Upload anything new (we are at the maximum)
Share this PDF (no secrets inside)	Share screenshots showing keys or tokens